

RaceIndyCar: Analysis of motorsport qualifying with a novel Python package

Anonymous
[Institution Name]

September 4, 2026

Abstract

Competitive motorsports are unique compared to other sports because, through qualifying, an advantage can be gained before the day of the event. However, the extent of qualifying's advantage can differ heavily across race series. To quantify these differences, lap-by-lap data were obtained using the Python libraries FastF1 and PyNascar. A new pip-installable package called RaceIndyCar was created in order to provide accessible access to IndyCar. The official IndyCar.com website was scraped for start position, finish position, average speed, and more for the package. Individual track section speeds and times are also included for every lap of every race since 2013. Then, for an analysis of all three series, two regression models were deployed. One uses driver quality and race-specific fixed effects, and the other adds the qualifying position to the same model. Expressed as partial R^2 , qualifying explains approximately 17% of the variation that driver quality does not already explain in F1, compared with 7% in IndyCar and 4% in NASCAR. This research shows that F1 teams should focus more on the strategy for qualifying as opposed to IndyCar and NASCAR teams, who should focus more on improving their car for race day, even if it means performing worse in qualifying.

Keywords: Motorsports, IndyCar, Formula 1, NASCAR, Qualifying, Sports Analytics

1 Introduction

Competitive motorsports are unique compared to other sports because, through qualifying, an advantage can be gained before the day of the event. Formula 1 (F1), NASCAR, and IndyCar all hold qualifying sessions that, on the surface, appear to have varying effects on the results of the race. For example, take two seven-time champions of their respective series: Lewis Hamilton (F1) and Jimmie Johnson (NASCAR). 61 of Lewis Hamilton's 106 race wins ([StatsF1 2026](#)) came from pole position, compared to a mere 15 of Jimmie Johnson's 83 wins ([MotorsportStats 2026](#)), pointing to potentially drastic differences in the relative importance of qualifying across race series.

So, we aim to quantify the relative importance of qualifying between these three major race series. In doing so, we create a new pip-installable Python package for obtaining lap-by-lap IndyCar data, which was previously unobtainable. Using lap-level information for each series, we adjust for driver performance to isolate the effects and try to gain actionable insight into the differences in race weekend operations across series.

2 RaceIndyCar

RaceIndyCar is a Python pip-installable package to obtain and analyze IndyCar data. By scraping information from the official [IndyCar](#) website, we can examine all weekend events and their corresponding sessions.

The specific information we have available depends on the session and release date. For example, not all races have the same number of practice sessions. Additionally, lap-by-lap data is only available for 2013 and beyond (with all data being from 1996-Present).

2.1 Data Collection

Data was obtained using the FastF1 and PyNascar Python libraries. The custom package described below, RaceIndyCar, was created to obtain lap-by-lap data for IndyCar. For each race, general statistics such as start position, finish position, average speed, and more are kept. Additionally, we obtain data for each lap of every race, including each driver's position, lap speed, and lap time.

In an IndyCar season, there are around 17 race events that occur. Each race event typically spans a weekend, where within that event there are individual sessions (such as practices, qualifying, and the race itself). We use this idea of events and sessions in our implementation:

```
import raceindycar

# Enable caching so downloaded and parsed data can be reused.
raceindycar.enable_cache(
    cache_dir=".cache/raceindycar",
    cache_format="pickle",
)

# Get all events for a given year
races = raceindycar.get_season_events(2025)

# Returns an Event with attributes for each schedule column.
# The second argument does a fuzzy name match to find an event,
# or use a round number/integer event id found in the schedule.
indy25 = raceindycar.get_event(2025, "Indianapolis 500")

# Identify all sessions (such as practices and qualifying) for the event.
session_info = indy25.sessions

# Get a specific session from an event.
# The session argument can either be an EventsSessionID or a SessionName string.
race_session = indy25.get_session(session="R")

# Load data for that session.
race_session.load()

# A Pandas DataFrame of race data, including start position,
# finish position, average speed, etc.
race_session.results.sample(n=3, random_state=314).iloc[:, :6]
```

	DriverUrl	DriversID	FirstName	LastName	CarNumber	EventsSessionsDetailsID
1	/Drivers/David-Malukas	4636	David	Malukas	4	118154
26	/Drivers/Rinus-VeeKay	4614	Rinus	VeeKay	18	118179
31	/Drivers/Kyle-Kirkwood	4623	Kyle	Kirkwood	27	118184

Source: [Article Notebook](#)

It is not required to get an event and then get a session from that event as shown above. We can go straight to a session for a weekend:

Source: [Article Notebook](#)

```
det25_race = raceindycar.get_session(
    2025,
    "Detroit Grand Prix",
```

```
session="R",
)
```

Source: [Article Notebook](#)

From a session, we can obtain the lap-by-lap data, with information on every section of an IndyCar track. Lap results from a given session are scraped from PDFs that are around 500 pages long, so loading laps can take anywhere from 30–100 seconds.

Source: [Article Notebook](#)

```
race_session.load(laps=True)

# A Pandas DataFrame of each lap, including sectional and position data
laps = race_session.laps
laps.sample(n=3, random_state=314).iloc[:, :6]
```

	Driver	DriverNumber	LapNumber	Position	LapSpeed	LapTime
3438	Nolan Siegel	6	169	14.0	221.034	40.7177
2035	Kyle Kirkwood	27	171	12.0	223.298	40.3049
4519	Conor Daly	76	58	2.0	217.388	41.4007

Source: [Article Notebook](#)

Additionally, there are some convenience filters for lap data for accessibility. Note that team names and drivers can either be a string or a list:

Source: [Article Notebook](#)

```
laps.pick_drivers(["12", "2"]) # Use car number or driver name.
laps.pick_teams("Team Penske")

laps.pick_wo_pit() # Excludes pit-road laps
laps.pick_quicklaps(threshold=1.07) # Laps within threshold of fastest.
laps.pick_fastest()
example = laps.pick_laps(range(1, 21))
example.sample(n=3, random_state=314).iloc[:, :6]
```

	Driver	DriverNumber	LapNumber	Position	LapSpeed	LapTime
4554	Conor Daly	76	9	11.0	113.103	79.5733
2212	Marcus Ericsson	28	15	5.0	217.838	41.3152
2647	Ed Carpenter	33	18	13.0	215.801	41.7051

Source: [Article Notebook](#)

2.2 Package Design and Usage

2.2.1 Installation

RaceIndyCar is pip installable, or available on [PyPI](#).

```
pip install raceindycar
```

Requires Python 3.10+. Dependencies ([pandas](#), [requests](#), [requests-cache](#), [pdfplumber](#), [matplotlib](#)) are installed automatically.

2.2.2 Plotting

RaceIndyCar also offers six plotting functions with preset colors for drivers and teams. The default arguments for all of these functions will work for the given IndyCar data, but can be manipulated to take in an arbitrary racing dataframe.

The first two plotting functions can be run for drivers of a specified race.

Source: [Article Notebook](#)

```
import matplotlib.pyplot as plt
from raceindycar import plotting

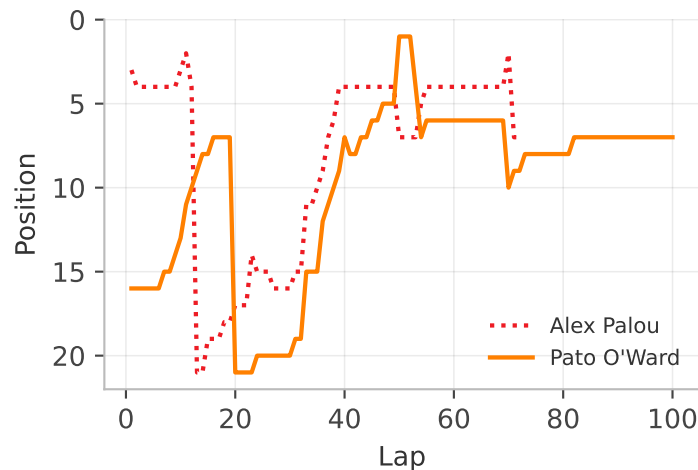
det25_race.load(laps=True)

drivers = ["10", "5"]
plotting.setup_mpl()

# Position-by-lap line chart
fig, ax = plotting.plot_position(
    det25_race,
    drivers=drivers,
)
fig.set_size_inches(4, 2.5)

# Lap time line chart
# fig, ax = plotting.plot_lap_times(
#     det25_race,
#     drivers=drivers,
# )

plt.show()
```



Source: [Article Notebook](#)

For example, `plot_qualifying_vs_finish` is shown here. In this case, Alex Palou crashed on lap 72, so he does not have a running position afterwards.

Additionally, we can plot information for multiple races:

Source: [Article Notebook](#)

```

import pandas as pd

det26_race = raceindycar.get_session(
    2026,
    "Detroit Grand Prix",
    session="R",
)

det26_race.load(laps=True)

combined_results = pd.concat([
    det25_race.results.assign(Year=2025),
    det26_race.results.assign(Year=2026),
])

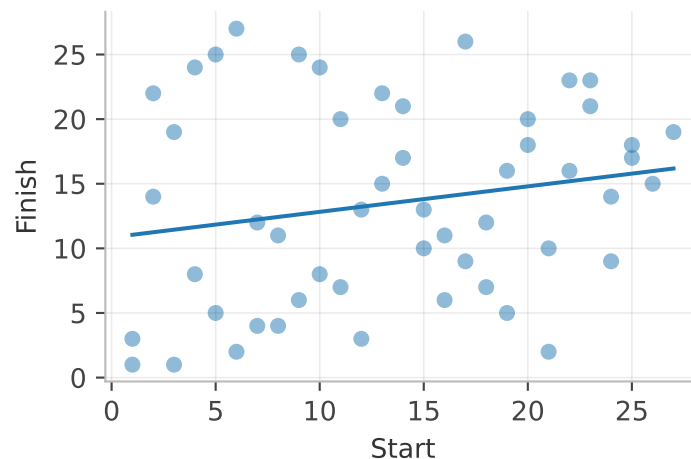
# Scatter of starting position vs finishing position with trend line
fig, ax = plotting.plot_qualifying_vs_finish(
    combined_results
)
fig.set_size_inches(4, 2.5)

# Histogram of positions gained/lost from start to finish
#fig, ax = plotting.plot_position_gain(
#    combined_results
#)

# Scatter of a results metric vs qualifying position, with trend line
#fig, ax = plotting.plot_metric_vs_qualifying(
#    combined_results,
#    metric_col="PointsEarned",
#)

plt.show()

```



Source: [Article Notebook](#)

For example, `plot_qualifying_vs_finish` is shown here.

2.2.3 Caching

This library mirrors [FastF1's Cache](#) API, and there are two stages for caching.

The first stage uses an SQLite table to store HTTP requests and results for [IndyCar](#).

The second stage stores Pickle/CSV files that contain information from loading a race.

Requests are rate limited to around one request per two seconds.

Source: [Article Notebook](#)

```
from raceindycar.cache import Cache

# Enable Cache with specified directory.
# Use either "pickle" or "csv" format.
raceindycar.enable_cache(
    cache_dir=".cache/raceindycar",
    cache_format="pickle",
)

# Wipe Stage 2 (parsed/pickle) data, keeping the HTTP cache.
# Optionally, wipe all Stage 1 HTTP requests with deep=True.
Cache.clear_cache(deep=False)

# Never hit the network and only use cached data,
# raising an error if the data is not found locally.
Cache.offline_mode(True)

# Temporarily bypass the cache.
with Cache.disabled():
    ...
```

Source: [Article Notebook](#)

2.2.4 Summary and Application

Through utilizing official [IndyCar data](#), RaceIndyCar obtains session-level data, even down to individual section times for every lap. We can store this information for reuse and plot key metrics.

Applying this at a weekend level, we can see how qualifying lap times associate with race lap times for individual drivers. On a multi-race level, we can start to identify patterns and trends to create actionable results, such as identifying optimal practice time, resource allocation to adjusting the car from qualifying to race day, and more.

3 Quantifying the Importance of Qualifying

Let Q_{ir} , F_{ir} denote qualifying and finishing position for driver i in race r , and $L_{ir\ell}$ the lap time on lap ℓ .

3.1 Pace Normalization

Raw lap times are first converted to a race/lap-normalized pace score,

$$z_{ir\ell} = \frac{L_{ir\ell} - \text{median}_j(L_{jr\ell})}{\text{sd}_j(L_{jr\ell})}, \quad (1)$$

making pace comparable across tracks and eras.

3.2 Driver and Team Quality

A race-level pace summary $\bar{z}_{ir} = \text{median}_\ell(z_{ir\ell})$ is then aggregated, leave-one-race-out, into a driver quality score

$$\text{DQ}_{ir} = -\frac{1}{N_i - 1} \sum_{r' \neq r} \bar{z}_{ir'}, \quad (2)$$

sign-flipped so higher is better, with team quality TQ_{ir} defined analogously. Excluding race r from its own quality estimate is essential: including it would let a driver's result in r partly explain itself, inflating quality's apparent explanatory power and artificially shrinking qualifying's incremental contribution. Both scores are then z-scored within series (DQ_{ir}^z , TQ_{ir}^z , sample sd) so coefficients are comparable across F1, IndyCar, and NASCAR despite each series having a different raw pace scale.

3.3 Regression Framework

The core analysis fits six nested OLS models per series, each with race fixed effects α_r to absorb track- and event-level heterogeneity (weather, field size, caution frequency) so that qualifying and quality are identified from within-race variation only:

$$\begin{aligned} F_{ir} &= \alpha_r + \beta_1 Q_{ir} + \varepsilon_{ir} & (\text{M1, qualifying only}) \\ F_{ir} &= \alpha_r + \gamma_1 \text{DQ}_{ir}^z + \varepsilon_{ir} & (\text{M2, driver quality only}) \\ F_{ir} &= \alpha_r + \beta_2 Q_{ir} + \gamma_2 \text{DQ}_{ir}^z + \varepsilon_{ir} & (\text{M3, both}) \\ F_{ir} &= \alpha_r + \delta_1 \text{TQ}_{ir}^z + \varepsilon_{ir} & (\text{M4, team quality only}) \\ F_{ir} &= \alpha_r + \beta_3 Q_{ir} + \delta_2 \text{TQ}_{ir}^z + \varepsilon_{ir} & (\text{M5, both}) \\ F_{ir} &= \alpha_r + \beta_4 Q_{ir} + \gamma_3 \text{DQ}_{ir}^z + \delta_3 \text{TQ}_{ir}^z + \varepsilon_{ir} & (\text{M6, full}) \end{aligned} \quad (3)$$

Reduced/full pairs (e.g. M2 vs. M3) are always fit on an identical common sample, so any R^2 difference reflects the added variable alone rather than sample drift.

3.4 Partial R^2

We report both the raw increment and the partial R^2 ,

$$\Delta R^2 = R_F^2 - R_R^2, \quad R_{\text{partial}}^2 = \frac{R_F^2 - R_R^2}{1 - R_R^2}, \quad (4)$$

with partial R^2 as the headline statistic because raw increments aren't comparable across series with different baseline R_R^2 — a series where quality already explains most of the variance has structurally less room left for qualifying to add to, and the partial- R^2 normalization removes that ceiling effect.

3.5 Race-Level Analysis

To check the pooled estimate isn't driven by a handful of races, each race is also fit independently,

$$F_{ir} = \alpha_r^{(0)} + \beta_r Q_{ir} + \varepsilon_{ir}, \quad (5)$$

yielding a *distribution* of race-level R_r^2 rather than one pooled number, alongside per-race and pooled Spearman's ρ as a distribution-free complement to the linear β_r . Linear OLS is preferred over rank-based or ordinal alternatives throughout because it yields additive, directly interpretable R^2 decompositions — the mechanism the incremental/partial- R^2 comparisons depend on — while Spearman correlation serves only as a robustness check on the same relationship, not a replacement for it.

4 Results

Through our method, we find that position is a meaningful but partial predictor of race outcome, and its importance varies sharply by series. After conditioning on race fixed effects and each driver’s underlying pace, qualifying position still adds real explanatory power in all three series, while the size of that effect differs by roughly a factor of two to three between F1 and NASCAR.

4.1 Qualifying and Finishing Position

First, we examine how qualifying position relates to finishing position across the three series. We also look at the amount of position changes that occur from qualifying to finish, which is a proxy for how much variance there is in the race itself.

Figure 1a shows that in F1, for the high starting positions, has a higher average finish position than the other two series. However, this advantage compared to IndyCar ends at around position 7. NASCAR’s curve is flatter compared to the other two series, indicating that average finish positions for many starting positions are similar.

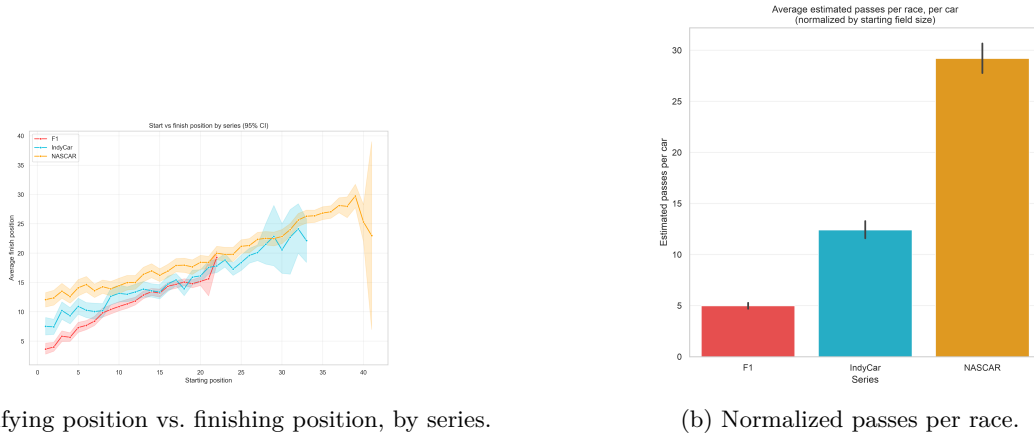


Figure 1: EDA of qualifying position vs. finishing position and passes per race

In addition, each NASCAR driver performs around 6 times as many passes as an F1 driver and over double the number for IndyCar drivers.

The scatter of qualifying position against finishing position (Figure 2a) shows a clear positive slope in all three series, but with very different amounts of scatter around it. F1’s cloud is tightest and most linear; IndyCar and NASCAR show substantially more vertical spread at every starting position, consistent with more frequent late-race reshuffling (strategy, cautions, pit-stop variance) loosening the grid-to-finish relationship. This is echoed in the position-gain histograms (Figure 2b): F1 drivers cluster tightly around “no change” from qualifying to finish, while IndyCar and especially NASCAR show wide, roughly symmetric distributions of gains and losses spanning ± 20 to ± 40 positions — a signature of much higher in-race variance.

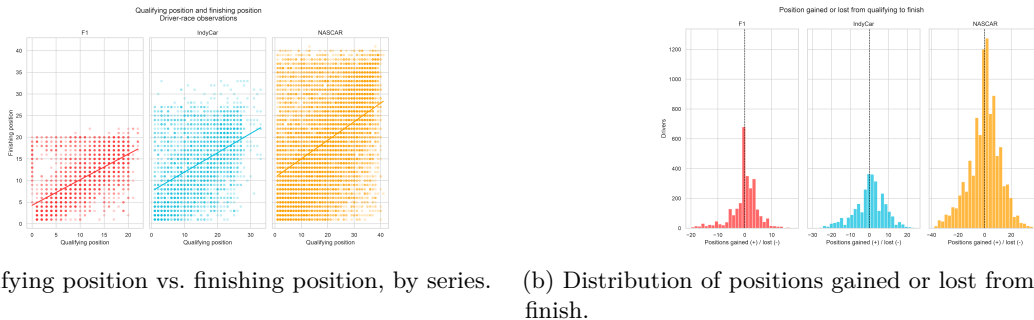


Figure 2: Raw relationship between starting and finishing position across series.

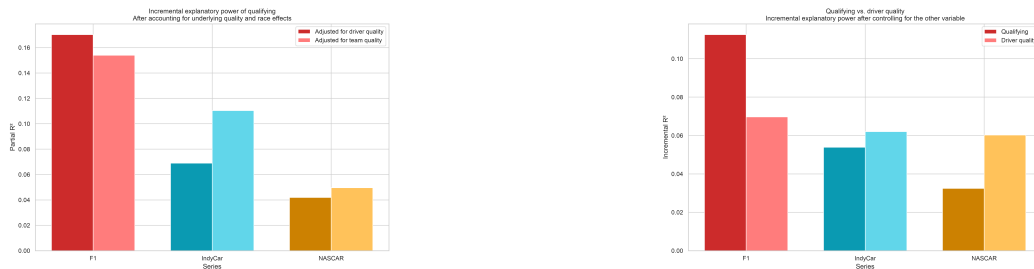
4.2 Adjusted Qualifying Importance

Using the common-sample specification (`finish ~ driver_quality_z + race` FE vs. `finish ~ qualifying + driver_quality_z + race` FE, identical observations in both models):

Table 1: Common-sample qualifying incremental/partial R^2 , controlling for driver quality and race fixed effects.

Series	n	Races	R^2 (quality only)	R^2 (+ qualifying)	Incremental R^2	Partial R^2
F1	2,778	142	0.339	0.451	0.113	0.170
IndyCar	2,908	113	0.218	0.272	0.054	0.069
NASCAR	9,695	272	0.226	0.258	0.032	0.042

Qualifying explains roughly **17% of the finishing-position variance left unexplained by driver quality in F1**, versus about **7% in IndyCar** and **4% in NASCAR**. The same ordering holds under the team-quality specification (Figure 3a): partial R^2 for qualifying is 0.154 (F1), 0.110 (IndyCar), and 0.050 (NASCAR). Figure 3b makes the driver-quality-vs-qualifying comparison directly: in F1, qualifying alone carries roughly as much unadjusted explanatory weight as driver quality; in IndyCar and NASCAR, driver quality’s incremental contribution meets or exceeds qualifying’s.



(a) Incremental explanatory power of qualifying, adjusted for driver and team quality. (b) Qualifying vs. driver quality: incremental R^2 controlling for the other variable.

Figure 3: Qualifying’s incremental explanatory power compared against underlying driver and team quality.

Figure 4 confirms why quality and qualifying are related but not redundant: driver quality (leave-one-race-out relative pace) correlates negatively with qualifying position in all three series (higher quality $\rightarrow$ better/lower grid slot), most steeply in NASCAR and IndyCar and more moderately in F1. Because faster drivers tend to qualify better, some of qualifying’s predictive power is really quality, which is exactly what the driver-quality control is designed to strip out. That qualifying still retains independent explanatory power after this adjustment (particularly in F1) indicates that grid position carries information beyond who’s fastest on a given weekend — most plausibly *track-position value*: the tactical advantage of racing near the front (clean air, fewer overtakes required, less exposure to pack incidents), which matters more where passing is harder (F1) and less where it’s easier or where race-day variance dominates (NASCAR, and to a lesser extent IndyCar).

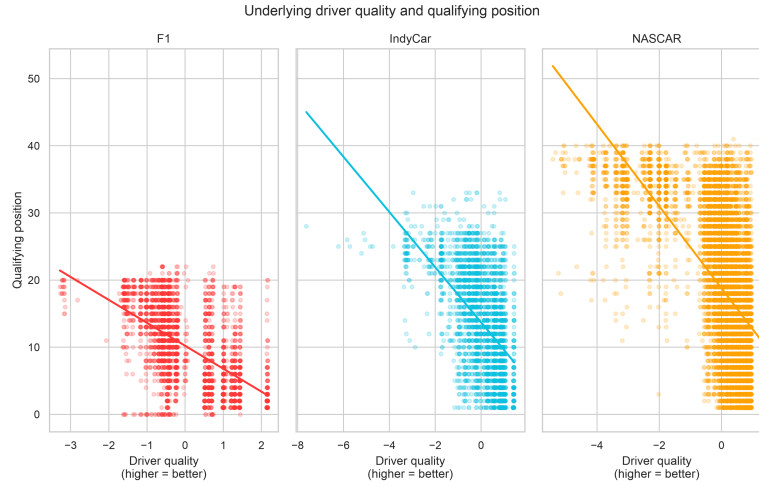


Figure 4: Underlying driver quality vs. qualifying position, by series.

4.3 Race-Level Variation

Figure 5 shows the distribution of race-level R^2 (a separate qualifying $\rightarrow$ finish regression fit within each individual race, no pooling). F1's median race-level R^2 (~ 0.41) is roughly double IndyCar's (~ 0.21) and more than double NASCAR's (~ 0.18), and F1's IQR sits noticeably higher on the axis. NASCAR shows the widest and most left-skewed spread, with many individual races near $R^2 = 0$, which is consistent with our EDA findings of frequent reshuffling and high in-race variance.

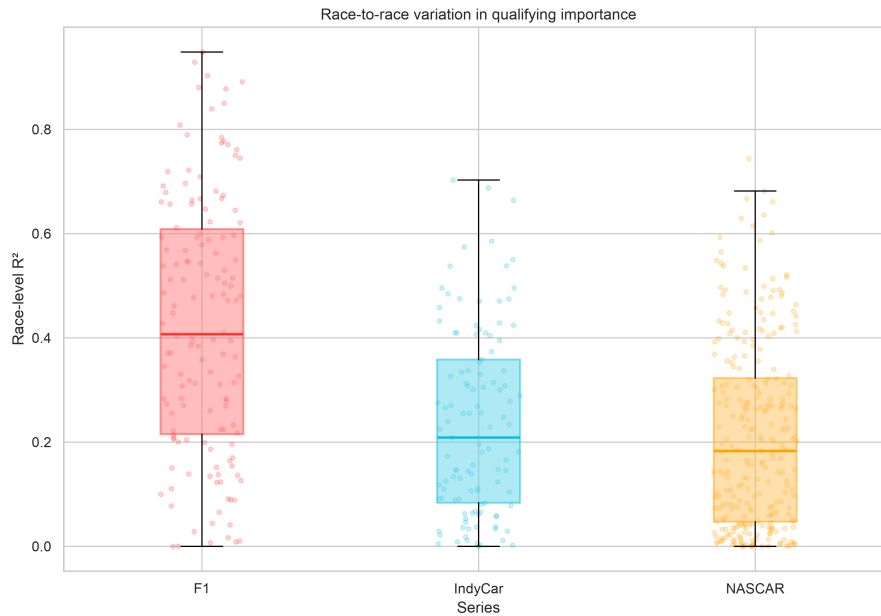


Figure 5: Race-to-race variation in qualifying importance (race-level R^2 , no pooling).

4.4 Qualifying and Lap 1 Position

As a secondary check, qualifying position correlates very strongly with observed Lap 1 running position (Pearson $r = 0.81$ F1, 0.92 IndyCar, 0.92 NASCAR), confirming grid position is a faithful proxy for actual race-start position across all three series. Using Lap 1 position instead of qualifying position as the predictor yields even larger partial R^2 values — 0.257 (F1), 0.087 (IndyCar), 0.048 (NASCAR) — suggesting the true track-position effect is,

if anything, slightly understated by using the qualifying result alone rather than the position a driver actually starts from once any grid penalties are applied.

5 Discussion

Qualifying proves to have statistically significant predictor of finish position, adjusting for race performance. However, this effect decreases in magnitude from F1 to IndyCar to NASCAR.

Motorsport viewers can take this information to inform decisions about what to watch on a race weekend. For F1 fans, compared to NASCAR or IndyCar fans, because 17% of the variability in finish position not explained by driver skill can be explained by qualifying position, it is more important to tune in to qualifying sessions because of its importance to race day. It also gives NASCAR and IndyCar fans more hope for their drivers who qualify poorly, as it only accounts for 4 and 6 percent of the aforementioned variability.

Additionally, these results can inform new viewers about their desired series to watch, and for the series themselves, what events they can market. If they enjoy watching qualifying and with that, a larger time investment in the sport, then perhaps F1 is the series to watch for them, as it has a larger effect on race day results than IndyCar or NASCAR. On the other hand, if they only want to watch an exciting race, then IndyCar and/or NASCAR is the way to go because there is more variance in finish position from start position, meaning there is more passing and more excitement for the viewer on a lap-by-lap basis. This is supported by the average number of passes for F1, IndyCar, and NASCAR races, which are 5, 12, and 29, respectively.

For the same reasons, F1 teams, relative to NASCAR and IndyCar teams, should be putting more effort into creating a solid qualifying performance because it has a larger effect on race results.

F1 drivers should be super aggressive on lap one. The difference in the predictive value for qualifying position and lap one position is 8.7%, whilst only being 1.8% and 0.6% for IndyCar and NASCAR. These drivers can take a more relaxed approach to the first lap, as it is not devastating to informing race results.

6 Limitations and Future Work

Differing weather conditions between qualifying and race day were not accounted for in this analysis. For the future, incorporating weather into the analysis can inform teams on how to adjust strategy based on varying conditions. We also did not create a function for teams to identify specifically the impact of adjustments for specific scenarios. For example, it would be helpful for a team to know if an x increase in race speed for a y position trade-off in qualifying will be worth it because they will gain back those lost positions.

7 Conclusion

Overall, this project creates new opportunities for data analytics in the IndyCar realm through the `raceindycar` package and offers a novel method of comparing races across series through our numerical analysis.

References

Source: [Article Notebook](#)

References

- MotorsportStats (2026), ‘Jimmie johnson — NASCAR cup series summary’, <https://motorsportstats.com/driver/jimmie-johnson/summary/series/nascar-cup-series>. Accessed 2026.
- StatsF1 (2026), ‘Lewis hamilton — pole positions and victories’, <https://www.statsf1.com/en/statistiques/pilote/pole/et-victoire.aspx>. Accessed 2026.